$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{h}{2^h} \right\rceil \leq \sum_{h=0}^{\infty} \left\lceil \frac{h}{2^h} \right\rceil = 2$$

and therefore, the total number of swaps is linear in the worst case (at most 2n).
So, our final analysis is that heapify's complexity is `O(n)`.

We leave the computation for a d-ary heap as an exercise: it is by all means similar to the one above, replacing the branching factor.

2.6.8 Beyond API methods: contains

One thing that heaps are definitely not good for is checking if an element is stored in them or not. We have no other choice than going through all the elements, until we find the one we were looking for, or we get to the end of the array. This means a linear time algorithm. Compare it with Hash Tables optimal average constant time, or even `O(log(n))` for binary search trees (average) or balanced binary search trees (worst case).

However, we also would like to support priority increment/decrement. As we have seen in section 2.6.5, for these operations efficiently retrieving the element whose priority needs to be changed is paramount. Therefore, when implementing heaps, we might add an auxiliary field, a `HashMap` from elements to positions, that allows us to check